Leader Election on a Unidirectional Ring with $1.5\,n \log n$ Messages

Algorithm B of Dolev, Klawe and Rodeh (1982)

Roman Volochai

4 November 2026

Distributed Systems

Outline

Problem and Algorithm A

The three improvements

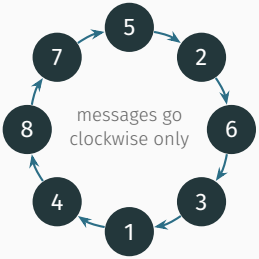
Algorithm B and the code

Example, phase by phase

Correctness and complexity

Problem and Algorithm A

The problem



Topology	ring of n processes, unidirectional: send to the right, receive from the left
Timing	asynchronous: arbitrary finite delays, but FIFO links
Knowledge	distinct ids, identical programs, n unknown
Goal	the owner of the maximum id becomes the leader
Cost	number of messages in the worst case

Burns 1980	Hirschberg, Sinclair 1980	Algorithm A	**Algorithm B**
lower bound $\Omega(n \log n)$	conjecture: one-way rings need $\Theta(n^2)$	$2n \log n + n$ (= Peterson 1982)	**$1.5\, n \log n + O(n)$** today

Algorithm A (Section 2): phases and local maxima



- FIFO links ⇒ the run splits into phases
- active processes form a smaller ring; passive ones only relay
- v cannot look to the right, so w decides: M1 brings v's value, M2 brings u's
- survivors = local maxima; each value moves to the next active process

Cost

$\leq$ half survive $\Rightarrow \leq \log n + 1$ phases. Every link carries one M1 and one M2 per phase:

$$\underbrace{n \log n}_{\text{M1}} + \underbrace{n \log n}_{\text{M2}} + n$$

Goal of Algorithm B and the road map (Section 3.1)

Goal

M1 must cross every link in every phase. Keep it. Send M2 over only $\approx$ half of the links: $n \log n + \frac{1}{2} n \log n = 1.5\, n \log n$.

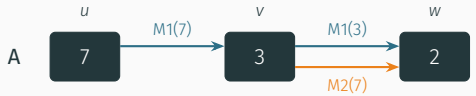
	Change	What the paper says about it
1	only a winner sends M2; new state WAITING	reduces M2, "by itself" not enough for $\frac{3}{2} n \log n$
2	passive processes drop hopeless M2s	reduces M2; behaviour of A unchanged
3	counter 2^p in M1	bounds every M2 by 2^p; changes the behaviour, needs a new proof

The three changes are applied to Algorithm A one after another; the result is Algorithm B.

The three improvements

Improvement 1: only a winner sends M2

Paper: v knows whether $\max(u) > \max(v)$ as soon as u's M1 arrives. First idea: send one bit instead of a number (fewer bits, same number of messages). Better: if $\max(u) > \max(v)$, w drops out whatever $\max(w)$ is, so v sends no M2.



w: 3 is not a local max ⇒ passive.
The M2 told w nothing new.

v sends no M2. w waits for 3, but the next message is an M1 ⇒ w becomes passive.
(v waits for 7: it moves on only if u won on its left.)

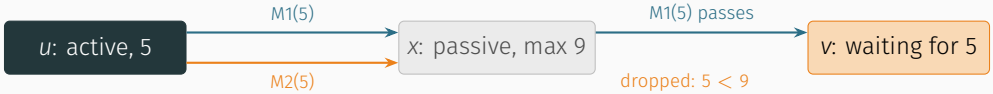
bigger value from the left ⇒ WAITING; smaller ⇒ PASSIVE and send $(2, \max(v))$.
WAITING + M2 ⇒ active next phase; two M1s in a row ⇒ passive.

Effect

Same survivors as A. Bit-reversal,
$n = 1024$: M2 10240 → 5120.

Improvement 2: passive processes drop hopeless M2s

Paper: let u, v be neighbouring active processes. If v survives, it will represent $\max(u)$. A passive x between them with $\max(x) > \max(u)$ knows that $\max(u)$ is not the global maximum, so it does not forward u's M2, and v becomes passive.



- passive processes remember $\max(x)$, the largest value that passed them in an M1
- the M1 must still pass: v needs it to decide about its own value, which may be the maximum
- key detail: B sends $(2, \max(v))$, not $(2, i)$ as A, so x can compare

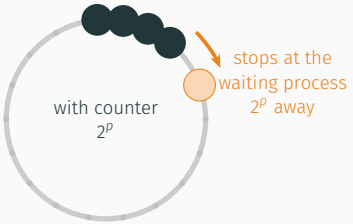
Effect

Random rings, $n = 1024$: M2 $3267 \to 2790$ (-15%).
Survivors are still the local maxima.

Improvement 3: the problem it solves

Paper: after a few phases the active processes may sit in a small part of the ring. Then all the savings happen there, while one M2 still travels almost the whole circle.



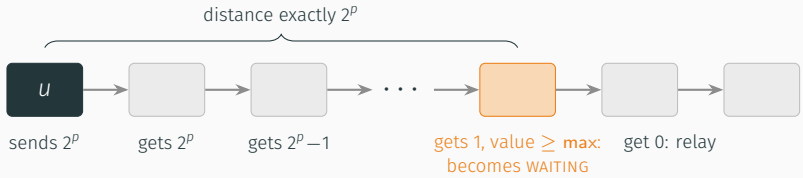


Aim stated in the paper: an M2 sent in phase p never travels farther than 2^p from its sender.

Crowded ring ($n = 1024$: ids 1..1008 in order, then 16 large ids together): M2 2048 $\rightarrow$ 1040.

Improvement 3: the counter 2^p

M1 of phase p carries $c = 2^p$; passive and waiting processes decrease it.



Guarantee

Waiting processes never forward M2 $\Rightarrow$ an M2 of phase p stops within 2^p: at a waiting process or at one with a bigger max.

Price (paper)

Passive processes can become active again, and survivors need not be local maxima: on ring $7, 3, 4, 5, 8, 2, 6, 1$ value 7 survives phase 1 although its active neighbour has 8.

What each improvement buys (measured)

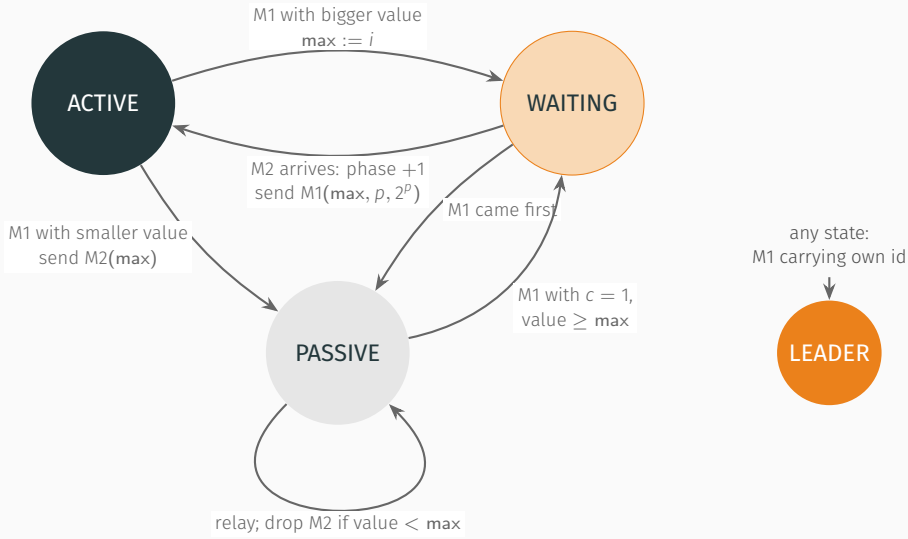
	bit-reversal		crowded		random (avg. 30)	
$n = 1024$	M2	total	M2	total	M2	total
A	10240	21504	4096	9216	6315	13653
A + 1	5120	16384	2048	7168	3267	10606
A + 1 + 2	5120	16384	2048	7168	2790	10129
B = A + 1 + 2 + 3	5120	15361	1040	6145	2002	11057

- improvement 1 halves M2 everywhere; improvement 2 helps when values are mixed
- improvement 3 is what protects the worst case (crowded ring: M2 halved again)
- honest detail: on random rings the counter adds phases ($7 \to 9$), so B pays more M1 than A+1+2; this is the price of the guarantee

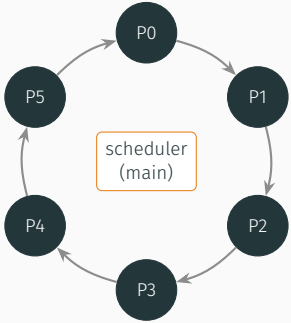
Every intermediate version simulated phase by phase; the B column agrees with the Go program.

Algorithm B and the code

Algorithm B: states and transitions (Section 3.2)



Implementation in Go: one goroutine per process



links: channels, buffer 1

Goroutine of a process (`node.run`)

```
for {
    select {
    case m := <-v.in:      // a message arrived:
        v.inbox = append(v.inbox, m)  // keep it
    case t := <-v.turn:    // the scheduler says go:
        v.step(algo, t)    // handle one event
    }
}
```

`step`: load state, take the oldest message, call `handleB`, send, save state.

Scheduler (Run)

```
for ready.size() > 0 {
    i := ready.random(rng)  // seeded random order
    time.Sleep(Delay)       // global delay (-delay)
    nodes[i].turn <- turn{...}
    r := <-done
}
```

Handling one message: `handleB`

```
func handleB(s State, m Message) (State, []Message) {
    // before the switch: an M1 carrying my own id -> leader
    switch s.Status {
    case Active:                                    // B1
        if m.Value > s.Max {                        // improvement 1
            s.Max, s.Status = m.Value, Waiting
            return s, nil
        }
        s.Status = Passive
        return s, []Message{newM2(s.Max)}           // (2, max(v))
    case Waiting:                                   // B2
        if m.Kind == M2 {
            s.Phase, s.Status = s.Phase+1, Active
            return s, []Message{newM1(s.Max, s.Phase, 1<<s.Phase)}
        }
        s.Status = Passive                          // two M1 in a row
        return passiveB(s, m)
    default:                                        // B3
        return passiveB(s, m)
    }
}
```

- a pure function: state and message in, new state and messages out
- cases are B1, B2, B3 of the paper
- `1<<s.Phase` is the counter 2^p

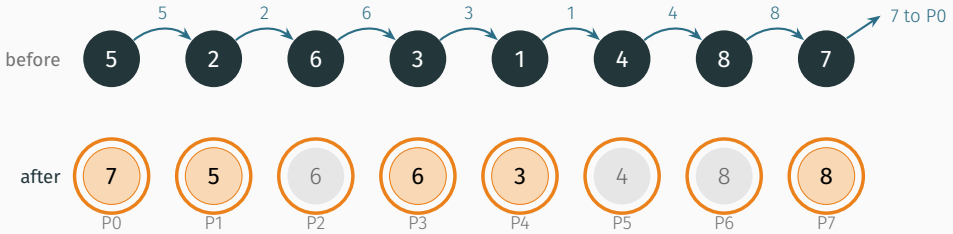
The passive process: passiveB

```
func passiveB(s State, m Message) (State, []Message) {
    if m.Kind == M2 {                                  // B3.2
        if m.Value < s.Max {
            return s, nil                              // improvement 2
        }
        return s, []Message{m}
    }
    if m.Value >= s.Max && m.Counter >= 1 {            // B3.1
        s.Max = m.Value
        if m.Counter > 1 {
            return s, []Message{newM1(m.Value, m.Phase, m.Counter-1)}
        }
        s.Status, s.Phase = Waiting, m.Phase           // improvement 3
        return s, []Message{newM1(m.Value, m.Phase, 0)}
    }
    return s, []Message{newM1(m.Value, m.Phase, 0)}
}
```

- M2: drop it if a bigger value was seen
- M1: remember the value and count down; at 1 become a candidate
- counter 0: plain relay

Example, phase by phase

Example 5, 2, 6, 3, 1, 4, 8, 7 · phase 0, step 1: everyone sends M1 (counter 1)



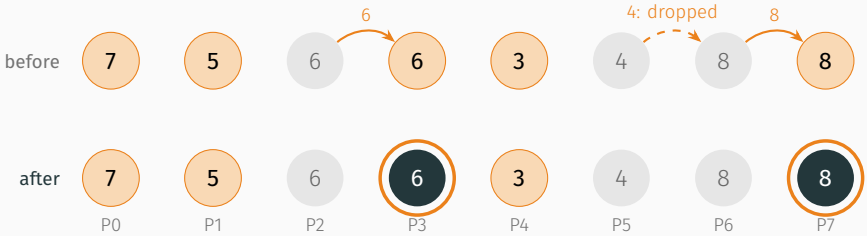
Each process compares the value from its left neighbour with its own:

- **bigger** ⇒ the left value may move to me: WAITING with it (P0, P1, P3, P4, P7)
- **smaller** ⇒ my value won on the left: PASSIVE, and I will send M2 with my value (P2, P5, P6)

active waiting passive number = max changed in this step

8 M1

Phase 0, step 2: only the winners send M2

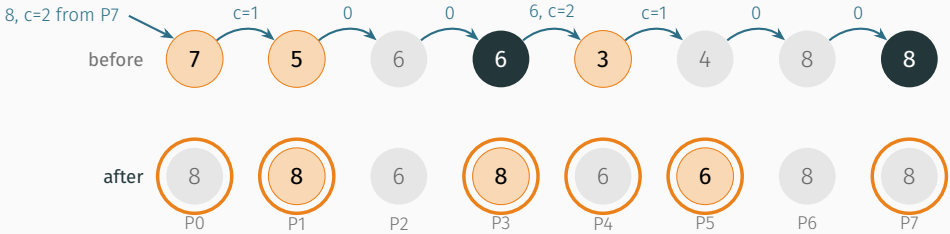


- M2(6): P2 → P3. P3 waited for 6 ⇒ ACTIVE in phase 1 (same for M2(8): P6 → P7)
- M2(4): P5 → P6. P6 is passive with **max** = 8 > 4 ⇒ dropped: 4 cannot be the maximum (impr. 2)
- P0, P1, P4 keep waiting, but no M2 is coming to them

active waiting passive number = **max** changed in this step

3 M2 (A: 8)

Phase 1, step 1: M1 with counter 2 creates candidates



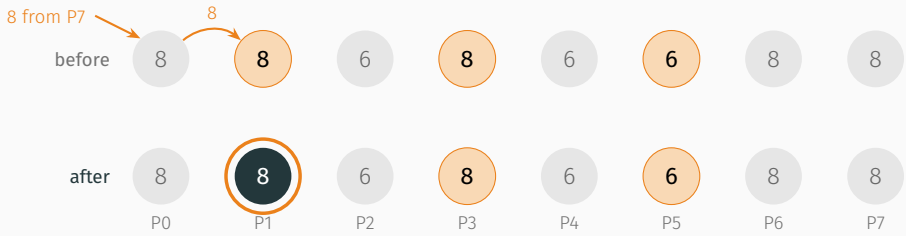
M1 always travels to the next active process; on the way the counter goes down.

- P7's M1(8): P0 and P1 got M1 instead of M2 ⇒ stop waiting; P1 gets c=1 ⇒ WAITING for 8; P3 (active, 6) gets $8 > 6$ ⇒ WAITING for 8
- P3's M1(6): P4 stops waiting; P5 gets c=1 ⇒ WAITING for 6; P7 (active, 8) gets $6 < 8$ ⇒ PASSIVE, sends M2(8)

active waiting passive number = **max** changed in this step

8 M1

Phase 1, step 2: M2(8) stops at the first waiting process

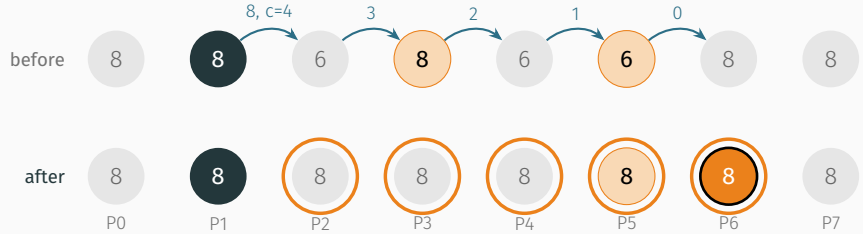


- M2(8): P7 → P0 → P1. P0 forwards it (its max is 8, not bigger). P1 waits for 8 ⇒ ACTIVE in phase 2
- a waiting process never forwards M2: it stops after 2 links $= 2^1$; it never reaches P3
- P3 (waiting for 8) and P5 (waiting for 6: 6 lost) get nothing; the next M1 makes them passive
- value 8 now sits at P1: $3 = 2^2 - 1$ steps after its owner P6

active waiting passive number = max changed in this step

1 M2, 2 links

Phase 2: one active process, the owner becomes the leader

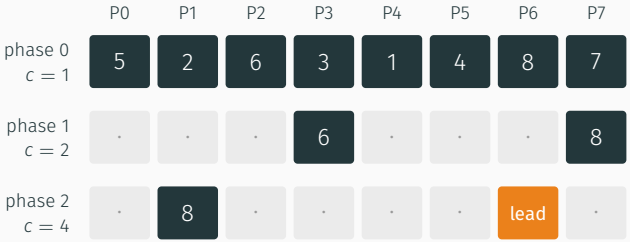


- P1 sends M1(8, c=4); passive processes store 8 and count down; P3 stops waiting; P5 gets c=1 (next candidate)
- P6 receives 8, its own id: the value went around without meeting another active process ⇒ P6 is the leader
- in total 21 M1 + 5 M2 = 26 messages; Algorithm A on the same ring: 40

active waiting passive number = **max** changed in this step

5 M1

The whole run at a glance



26 messages (21 M1 + 5 M2); on the same ring A sends 40, A+1 and A+1+2 send 31. Value 8 sits $0, 1, 3 = 2^p - 1$ steps after its owner P6.

The same run in the program's log

```
$ go run . -ids 5,2,6,3,1,4,8,7 -seed 3
step   22  P5 (id 4) got M1(value=6, phase=1, counter=1)  -> WAITING max=6 phase=1
step   24  P1 (id 2) got M1(value=8, phase=1, counter=1)  -> WAITING max=8 phase=1
step   27  P7 (id 7) got M1(value=6, phase=1, counter=0)  -> PASSIVE max=8 phase=1
           P7 sent M2(value=8) to P0
step   28  P0 (id 5) got M2(value=8)                      -> PASSIVE max=8 phase=0
           P0 sent M2(value=8) to P1
step   29  P1 (id 2) got M2(value=8)                      -> ACTIVE max=8 phase=2
           P1 sent M1(value=8, phase=2, counter=4) to P2
...
step   34  P6 (id 8) got M1(value=8, phase=2, counter=0)  -> LEADER max=8 phase=0
           *** P6 (id 8) is the leader ***

phases 3   M1 21   M2 5   election total 26   bound 1.5 n log n + 8/3 n = 57.3
```

`go test` checks automatically: the right leader for every $n = 1, \ldots, 70$; fewer than $1.5\,n \log n + \frac{8}{3}n$ messages; and, on random rings, the facts used in the proof (values jump exactly 2^p per phase, at most $\approx n/2$ M2 per phase).

Correctness and complexity

Invariants (Section 3.3)

Notation: $A(p)$ = processes active at the start of phase p; $p\text{-}\max(v)$ = their values; $\mathrm{origin}(j)$ = the process with id j.

Numbers such as 3.3.1 are the numbering in the paper, so the proofs are easy to find there.

3.3.1 Distinct values

Values in $A(p)$ are distinct: each M2 carries one value and activates at most one process.

3.3.2 Short M2s

An M2 of phase p travels $\leq 2^p$ links: stopped by the process waiting at 2^p, or by the next active one.

3.3.5 The maximum survives

Some process of $A(p)$ holds the maximum M: nobody can drop M2(M), and by FIFO it reaches the waiting process before the next M1.

Unique leader

An M1 returns to its owner only if no other active process stopped it $\Rightarrow$ its value is M.

The key lemma: values move on a fixed timetable

Lemma 3.3.6 (the timetable)

Let $v \in A(p)$ and $j = p\text{-}\max(v)$. Then (a) $d(\mathrm{origin}(j), v) = 2^p - 1$; (b) every process from $\mathrm{origin}(j)$ to v has $p\text{-}\max(\cdot) = j$; (c) $d(w, v) \geq 2^p$ for every other $w \in A(p)$.

Proof idea (induction on p). v was activated by the M2 of $u \in A(p-1)$, so $d(u, v) \leq 2^{p-1}$ (3.3.2). Also $d(u, v) \geq 2^{p-1}$: by (c) if v was active, otherwise v started waiting through the counter, at exactly 2^{p-1}. Hence $(2^{p-1} - 1) + 2^{p-1} = 2^p - 1$. A closer active process would hold the same value, contradicting 3.3.1. □



each surviving value owns a stretch from its owner to its holder; stretches are disjoint

Corollary 3.3.7 (few active processes)

$|A(p)| \leq n/2^p \Rightarrow$ at most $\log n + 1$ phases.

Counting M2: runs are followed by silence

Lemma 3.3.8 (silence after every run of M2)

Let u send an M2 in phase p, f the last process forwarding it, and $d(f, g) = 2^p - 1$. Then no process in $(f, g]$ sends an M2 in phase p.



Proof idea. Let w be the next active process after u. If w is weaker, it waits (no M2), and the next one is $\geq 2^{p+1}$ from u by 3.3.6(c). If w holds $j > p\text{-}\max(u)$, u's M2 stops before $\mathrm{origin}(j)$, which is $2^p - 1$ before w by 3.3.6(a).

Corollary 3.3.9 (about $n/2$ M2 per phase)

In phase p at most $n \cdot 2^p/(2^{p+1} - 1) \approx n/2$ messages M2.

The message bound

Theorem 3.3.10 (the message bound)

Algorithm B sends fewer than $\frac{3}{2}\, n \log n + \frac{8}{3}\, n$ messages.

$$\#\mathrm{M1} \le n\,(\log n + 1)$$

$$\#\mathrm{M2} \le \sum_p \frac{n\,2^p}{2^{p+1}-1} = \sum_p \left(\frac{n}{2} + \frac{n}{2(2^{p+1}-1)}\right) < \tfrac{1}{2}\, n \log n + \tfrac{5}{3}\, n$$

M1: one per link per phase, at most $\log n + 1$ phases (Cor. 3.3.7). M2: Cor. 3.3.9 summed over all phases but the last, which has no M2.

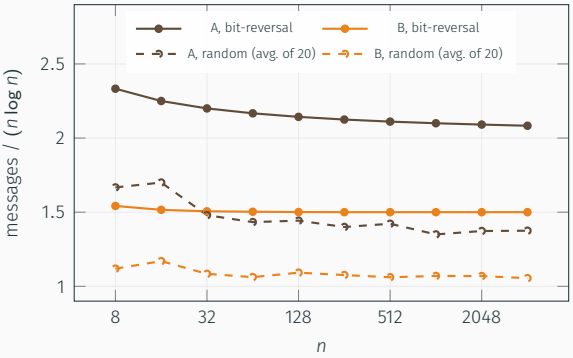
Constant at $n \log n$

$$2 \longrightarrow 1.5$$

The price

Every M1 carries a $\log n$-bit counter: in *bits*, B is worse than an optimised A.

Measurements: the constant 1.5 is attained



- **bit-reversal ids**: exactly $\log n + 1$ phases; B sends exactly $\frac{3}{2} n \log n + 1$, A exactly $2n \log n + n$
- only the $\frac{8}{3} n$ term of the bound is loose
- **random ids**: both far below the worst case

`go run . -bench`; table in the report

Conclusions

- Algorithm B = Algorithm A + three changes, all aimed at M2:
  1. only a winner sends M2 (new state WAITING)
  2. passive processes drop hopeless M2s (M2 carries $\max(v)$)
  3. a 2^p counter: no M2 goes further than 2^p (protects the worst case)
- Key invariant: values jump exactly 2^p in phase $p \Rightarrow$ active processes are 2^p apart and M2 runs are separated by silence
- Result: $< \frac{3}{2}\, n \log n + \frac{8}{3}\, n$ messages; the constant 1.5 is attained

Questions?

References

- D. Dolev, M. Klawe, M. Rodeh. An $O(n \log n)$ unidirectional distributed algorithm for extrema finding in a circle. *Journal of Algorithms* 3(3):245–260, 1982.
- G. L. Peterson. An $O(n \log n)$ unidirectional algorithm for the circular extrema problem. *ACM TOPLAS* 4(4):758–762, 1982.
- D. S. Hirschberg, J. B. Sinclair. Decentralized extrema-finding in circular configurations of processors. *Communications of the ACM* 23(11):627–628, 1980.
- J. E. Burns. A formal model for message passing systems. Technical Report TR-91, Indiana University, 1980.

Code, report and slides:
`github.com/RomanVolochai/distributed-systems-unidir-leader-finder-b`